

# SOFTWARE DESIGN AND ARCHITECTURE SWE 4601

## *Facebook was once an empty text file on someone's computer*

We need to understand how to-

- Architect a system to serve the needs of its users
- Write code that's easy to change
- Write code that's easy to maintain
- Write code that's easy to test

*The goal of software is to continually produce something that satisfies the needs of its users, while minimizing the effort it takes to do so.*

Software that doesn't serve the **needs** of its users, simply isn't good software.

And since the needs of our users changes often, it's important to make sure that software was **designed** in order to be **changed**.

If software cannot be changed (easily), that makes it bad software, because it prevents us from satisfying the current needs of our users.

# What is Software Design?

Software Design is the process of planning how a software system will work before writing the actual code. It focuses on **how** the software will be structured and how its parts will interact.

*Drawing a blueprint for a house before building it*

**Who are the designers?** - Software engineers or architects who **create the system structure**. They decide on: The software architecture. Modules, classes, functions, data structures and algorithms

**Who are the manufacturers?** - The programmers or developers. They write the actual code.

**What is the product?** - The working software application that users interact with.

**What is the building process?** - SDLC Life Cycle

# What is Software Architecture?

Software Architecture is the **high-level** structure of a software system. It defines the main components, how they interact, and the rules for designing and evolving the system.

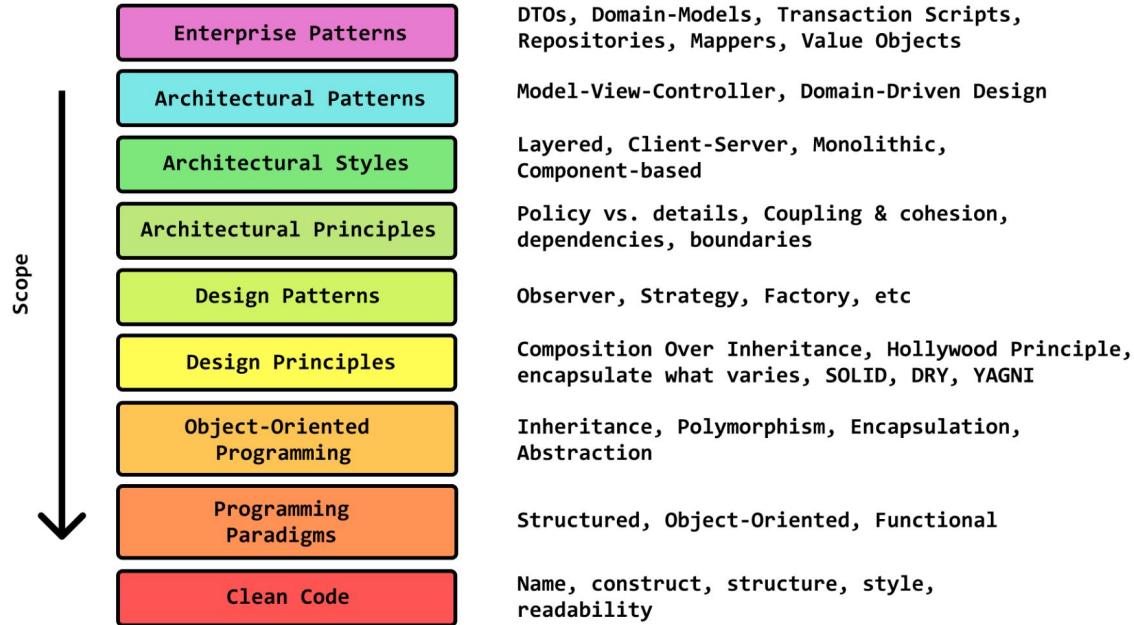
*Like the overall floor plan of a building – showing rooms, connections, and materials used.*

**Who are the architects?** - Software architects or experienced engineers who define the system's structure. They make decisions about: Main **components** || **Communication** between components || **Technology** stack (e.g., databases, frameworks) || **Quality** attributes (performance, security, scalability)  
**What are the components?** - UI Layer (Frontend) || Business Logic Layer || Data Layer (Database) || APIs and External Services

**What is the product?** - A well-structured system that can **grow** and **change** easily while meeting business needs.

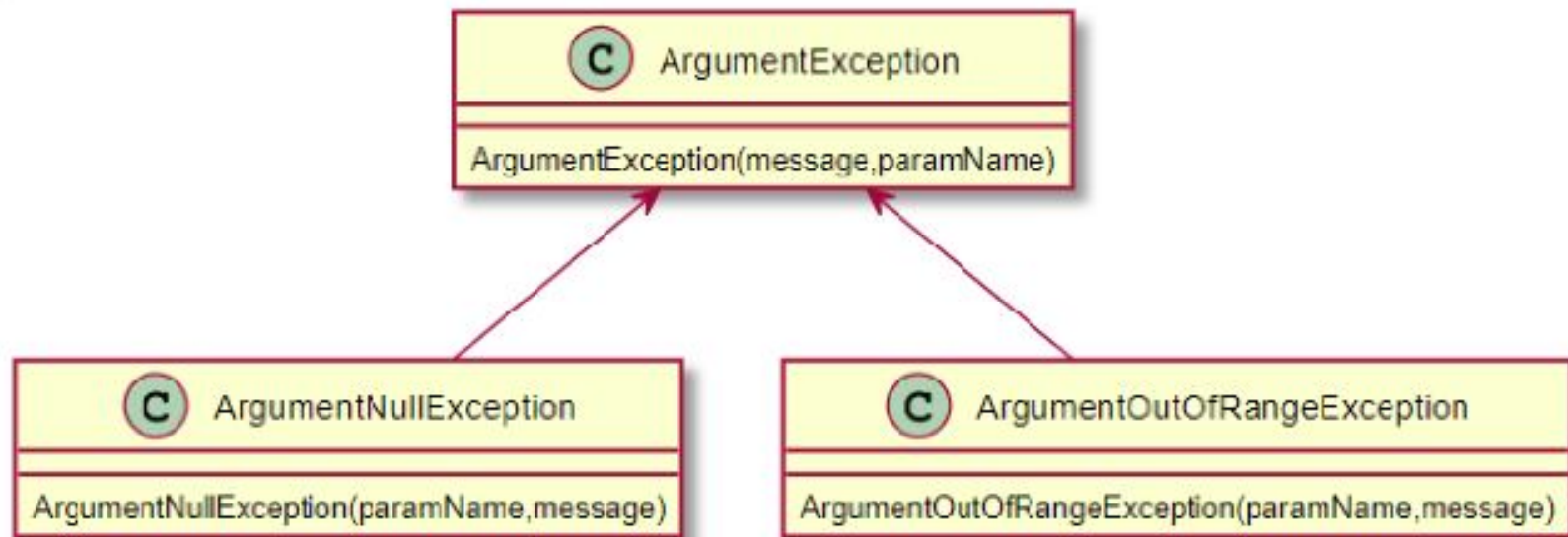
**What is the process?** - Define **architecture patterns** (e.g., layered, microservices), then design and validate them before coding.

# The stack





# POLA





← → ↺ rokomari.com/book/ebook

🔍 Hello, Sign in

Search by superstores (Umbrella, Tangram, Casio)

(Showing 1 to 60 of 10000 eBooks)

**By Authors**

☐ Rabindranath Tagore

☐ Saratchandra Chattopadhyay

☐ William Shakespeare

☐ Bibhutibhusan Bandopadhyay

☐ Sunil Gangapadhyay (Nillohit)

☐ Jules Verne

☐ Sir Arthur Conan Doyle

☐ Sukumar Rnv

**By Publishers**

☐ Muthoboi

**ইতিকাক**  
সাদিক ফারহান  
TK. 45

**সবুজ পাতার বন**  
শাইখ আব্দুল আযীয আত তারিফী  
TK. 101

**পিঠাপিড়িয়া**  
রূপচাঁদা  
TK. 0

**ইউ মাস্ট ডু বিজনেস**  
শাইখ ড. তাওফিক চৌধুরী  
TK. 27

**সারভাইভাল সার্ভিস**  
সাদিক ফারহান  
TK. 45

**QNA**  
সামিউইল  
TK. 101

**ইলেকট্রনিক সেমস**  
কম্পিউটার সেটের  
মহামারী  
TK. 101

**বিজনেস ব্লুপ্রিন্ট**  
কোচ কাফান  
TK. 101

# SRP

```
public class Car {  
    private double fuelLevel;  
  
    public Car(double fuelLevel) {  
        this.fuelLevel = fuelLevel;  
    }  
  
    public void drive(double distance) {  
        if (this.fuelLevel >= distance) {  
            this.fuelLevel -= distance;  
            System.out.printf("Driving %.2f miles. Fuel level: %.2f%n",  
distance, this.fuelLevel);  
        } else {  
            System.out.println("Not enough fuel");  
        }  
    }  
  
    public void fillGasTank(double amount) {  
        this.fuelLevel += amount;  
        System.out.printf("Adding %.2f gallons of gas. Fuel level: %.2f  
%n", amount, this.fuelLevel);  
    }  
}
```

# SRP

```
public class Car {  
    private FuelTank fuelTank;  
  
    public Car(FuelTank fuelTank) {  
        this.fuelTank = fuelTank;  
    }  
  
    public void drive(double distance) {  
        if (this.fuelTank.hasEnoughFuel(distance)) {  
            this.fuelTank.consumeFuel(distance);  
            System.out.printf("Driving %.2f miles. Fuel level: %.2f%n",  
distance, this.fuelTank.getFuelLevel());  
        } else {  
            System.out.println("Not enough fuel");  
        }  
    }  
}
```

# SRP

```
public class FuelTank {  
    private double fuelLevel;  
  
    public FuelTank(double fuelLevel) {  
        this.fuelLevel = fuelLevel;  
    }  
  
    public boolean hasEnoughFuel(double distance) {  
        return this.fuelLevel >= distance;  
    }  
  
    public void consumeFuel(double distance) {  
        this.fuelLevel -= distance;  
    }  
  
    public double getFuelLevel() {  
        return this.fuelLevel;  
    }  
}
```

```
enum EmployeeType {  
    Permanent, Contractual, PartTime  
}
```

```
class Employee {  
    private readonly EmployeeType type;  
    private int transportAllowance;  
  
    public Employee(EmployeeType type, int transportAllowance) {  
        this.type = type;  
        this.transportAllowance = transportAllowance;  
    }  
  
    public int CalculateSalary(int basic, int phr, int hw) {  
        if (type == EmployeeType.Permanent) {  
            return basic + basic / 4 + transportAllowance;  
        } else if (type == EmployeeType.Contractual) {  
            return basic + transportAllowance + phr * hw;  
        } else {  
            return basic + phr * hw;  
        }  
    }  
}
```

```
    public int CalculateBonus(int basic) {  
        if (type == EmployeeType.Permanent) {  
            return basic / 2;  
        } else if (type == EmployeeType.PartTime) {  
            return 0;  
        } else {  
            return 5000;  
        }  
    }  
}
```

```
}
```

```
enum EmployeeType {
    PERMANENT, CONTRACTUAL, PARTTIME
}

abstract class Employee {
    protected int transportAllowance;

    public Employee(int transportAllowance) {
        this.transportAllowance = transportAllowance;
    }

    public abstract int calculateSalary(int basic, int phr, int hw);

    public abstract int calculateBonus(int basic);
}

class ContractualEmployee extends Employee {
    public ContractualEmployee(int transportAllowance) {
        super(transportAllowance);
    }

    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + transportAllowance + phr * hw;
    }

    @Override
    public int calculateBonus(int basic) {
        return 5000;
    }
}
```



```
class Vehicle {  
    public void startEngine() {  
        // Logic to start the engine  
    }  
  
    public void drive() {  
        // Logic to drive  
    }  
  
    public void stop() {  
        // Logic to stop  
    }  
}  
  
class Truck extends Vehicle {  
    // Inherits the startEngine, drive, and stop methods  
}  
  
class Bicycle extends Vehicle {  
    @Override  
    public void startEngine() {  
        throw new UnsupportedOperationException("Bicycles don't have  
engines!");  
    }  
}
```

```
abstract class Vehicle {  
    public abstract void drive();  
    public abstract void stop();  
}
```

```
class MotorVehicle extends Vehicle {  
    public void startEngine() {  
        // Logic to start the engine  
    }  

```

```
    @Override  
    public void drive() {  
        // Logic to drive  
    }  

```

```
    @Override  
    public void stop() {  
        // Logic to stop  
    }  
}
```

```
class MotorVehicle extends Vehicle {  
    public void startEngine() {  
        // Logic to start the engine  
    }  

```

```
    @Override  
    public void drive() {  
        // Logic to drive  
    }  

```

```
    @Override  
    public void stop() {  
        // Logic to stop  
    }  

```

```
} class Bicycle extends Vehicle {  
    @Override  
    public void drive() {  
        // Logic to pedal the bicycle  
    }  

```

```
    @Override  
    public void stop() {  
        // Logic to stop the bicycle  
    }  

```

```
}
```



```
class Machine:
    def print(self, document):
        pass

    def scan(self, document):
        pass

    def fax(self, document):
        pass
```

*# OldPrinter can only print, but is forced to implement scan and fax*

```
class OldPrinter(Machine):
    def print(self, document):
        print("Printing:", document)

    def scan(self, document):
        raise NotImplementedError("OldPrinter does not support scanning")

    def fax(self, document):
        raise NotImplementedError("OldPrinter does not support faxing")
```

*# Break the interface into smaller, specific ones*

```
class Printer:
```

```
    def print(self, document):  
        pass
```

```
class Scanner:
```

```
    def scan(self, document):  
        pass
```

```
class Fax:
```

```
    def fax(self, document):  
        pass
```

*# Only implements what it needs*

```
class SimplePrinter(Printer):
```

```
    def print(self, document):  
        print("Printing:", document)
```

```
class MultiFunctionPrinter(Printer, Scanner, Fax):
```

```
    def print(self, document):  
        print("Printing:", document)
```

```
    def scan(self, document):  
        print("Scanning:", document)
```

```
    def fax(self, document):  
        print("Faxing:", document)
```

```
class MySQLDatabase:
    def connect(self):
        print("Connected to MySQL database")

class App:
    def __init__(self):
        self.db = MySQLDatabase() # App depends directly on a low-level detail

    def start(self):
        self.db.connect()
```

You can't easily switch to PostgreSQL or mock it for testing.

```
# Define an abstraction (interface)
class Database:
    def connect(self):
        pass

# Low-level module implements the abstraction
class MySQLDatabase(Database):
    def connect(self):
        print("Connected to MySQL database")

# High-level module depends on the abstraction, not the concrete class
class App:
    def __init__(self, db: Database):
        self.db = db # Depends on abstraction

    def start(self):
        self.db.connect()

# Dependency injection: provide the actual implementation
db = MySQLDatabase()
app = App(db)
app.start()
```

## Violation

```
def calculate_area_of_square(side):  
    return side * side  
  
def print_area_of_square(side):  
    area = side * side # Repeated logic  
    print("Area is:", area)
```

## Good Approach

```
def calculate_area_of_square(side):  
    return side * side  
  
def print_area_of_square(side):  
    area = calculate_area_of_square(side)  
    print("Area is:", area)
```

## Violation

```
def add(a, b):  
    # Complex and unnecessary abstraction  
    operations = {  
        'add': lambda x, y: x + y  
    }  
    return operations['add'](a, b)
```

## Good Approach

```
def add(a, b):  
    return a + b
```

```
class User
{
    private string _username;
    private string _password;
    private string _phoneNumber;    // Not needed
    private string _address;        // Not needed
    private bool _twoFactorEnabled; // No 2FA yet
    private string _profilePicture; // No UI yet

    public User(string username, string password)
    {
        _username = username;
        _password = password;
        _twoFactorEnabled = false; // Premature feature
    }
}
```

```
public bool Authenticate(string inputPassword)
{
    return _password == inputPassword;
}
```

*// Premature methods for unused features*

```
public void EnableTwoFactor() { _twoFactorEnabled = true; }
public void SetProfilePicture(string picturePath) { _profilePicture = picturePath; }
public void SetAddress(string address) { _address = address; }
public void SetPhoneNumber(string phone) { _phoneNumber = phone; }
}
```



```
class User
{
    private string _username;
    private string _password;

    public User(string username, string password)
    {
        _username = username;
        _password = password;
    }

    public bool Authenticate(string inputPassword)
    {
        return _password == inputPassword;
    }
}
```

